to the browser, and you could cause invalid data to be sent. You can generally read from the `Response` object safely, such as to inspect the final `StatusCode` or `ContentType` of the response.

Another common use for the `Use` extension method is to modify every request or response that passes through it. For example, you should send various HTTP headers with all your applications for security reasons. These headers often disable old, insecure legacy behaviors by browsers or restrict the features enabled by the browser. You learned about the HSTS header in chapter 28, but you can add other headers for additional security.

TIP You can test the security headers for your app at https://securityheaders.com, which also provides information about what headers you should add to your application and why.

Imagine you've been tasked with adding one such header—`X-Content-Type-Options: nosniff`, which provides added protection against cross-site scripting (XSS) attacks—to every response generated by your app. This sort of cross-cutting concern is perfect for middleware. You can use the `Use` extension method to intercept every request, set the response header, and then execute the rest of the middleware pipeline. No matter what response the pipeline generates, whether it's a static file, an error, or a Razor Page, the response will always have the security header.

Listing 31.4 shows a robust way to achieve this. When the middleware receives a request, it registers a callback that runs before Kestrel starts sending the response back to the browser. It then calls `next()` to run the rest of the middleware pipeline. When the pipeline generates a response, likely in some later middleware, Kestrel executes the callback and adds the header. This approach ensures that the header isn't accidentally removed by